



Digital IC Design & Verification
Physical Design
Lab Manual # 01 | Synthesis

Release: 2.1

Date: 23-June-2026

NUST Chip Design Centre (NCDC), Islamabad, Pakistan



Copyrights ©, NUST Chip Design Centre (NCDC). All Rights Reserved. This document is prepared by NCDC and is for intended recipients only. It is not allowed to copy, modify, distribute or share, in part or full, without the consent of NCDC officials.

Revision History

Revision Number	Revision Date	Revision By	Nature of Revision	Approved By
1.0	20/03/2025	Hira Sohail	Complete manual	Sharjeel Khilji
2.0	14/05/2026	Hira Sohail Fawad Ahmed	- Transform into Script-Based Synthesis from GUI based - Addition of Physical-Aware Synthesis	-
2.1	23/06/2026	Fawad Ahmed	Addition of Formal Verification i.e Logic Equivalence Check (LEC)	-



Table of Contents

Objective	3
Tools	4
Instructions	4
1. Introduction	4
2. Learning Objectives	5
3. The Paradigm Shift: Logical vs. Physical Synthesis	5
4. The Anatomy of Synthesis: Understanding the Input Files	6
4.1. The Automation Engine: TCL Scripts (.tcl)	7
4.2. The Design Rules: SDC Constraints (.sdc)	10
4.2.1. Clock Definition	10
4.2.2. Clock Uncertainty (Safety Margins)	10
4.2.3. Environment Modeling	11
4.2.4. I/O Timing Budgeting	12
4.2.5. DRV Constraints	12
4.3. The Physical Reality: MMMC Setup (.view)	13
4.4. The Building Blocks: Technology Libraries (.lib, .lef, .captable)	14
5. The Outputs of Synthesis	15
5.1. The Gate-Level Netlist (.v)	15
5.2. The Standard Delay Format (.sdf)	16
5.3. The Layout Databases (.db)	17
5.4. The Synthesis Reports (.rpt)	18
5.5. Formal Verification Scripts and Logs (.do / .log)	18
6. The Engine in Action: Executing Synthesis with Cadence Genus	18
6.1. Invoking the Cadence Environment	18
6.2. Invoking the Cadence Genus	18
6.3. Genus GUI	19
6.4. Output Files	20
7. Gate-Level (Post Synthesis) Simulation	21
8. Formal Verification (Logic Equivalence Checking)	22
8.1. Executing Cadence Conformal	22
8.2. Deep Dive into Run Results: Check 2 (Mapped vs. Optimized)	22
8.2.1. The Handshake (Verification Data)	22
8.2.2. The Proof of Optimization	23
8.2.3. The Mathematical Proof (The Compare Table)	23
8.2.4. The Final Sign-Off (The Dashboard)	23
8.2.5. Visual Debugging: Exploring the Conformal GUI	24



9. Design for Test (DFT)	24
9.1. Flow of DFT	25
9.2. The Anatomy of DFT: Building the "Safety Net" for Silicon	25
9.2.1. DFT Automation & Scan Chain Insertion	25
9.2.2. DFT Environment: The Test SDC (.sdc)	25
9.2.3. DFT Handoff: Reports & Exports	26
9.2.4. DFT in the Multi-Mode Multi-Corner (MMMC) Reality	26
9.3. Executing Synthesis with DFT	27
9.3.1. Invoking Cadence Genus	27
9.3.2. Output Files	27
9.4. Gate-Level (Post-Simulation) with DFT	28
10. Lab Task	29
11. Resources	30



Objective

The objective of this lab is to

- Synthesize **RTL into a gate-level netlist** using Cadence Genus, differentiating between purely Logical and Physical-Aware (iSpatial) synthesis flows.
- Configure a **Multi-Mode Multi-Corner (MMMC)** timing environment to optimize both high-speed functional operation and slow-speed scan shifting simultaneously.
- Insert **Design-for-Test (DFT)** scan chains into the design and analyze the resulting physical area and timing penalties.
- Simulate the post-synthesis netlist using Standard Delay Format (SDF) in Cadence Xcelium to verify physical timing and functional correctness.
- Perform **Formal Verification (LEC)** using Cadence Conformal to mathematically prove that the structural gate-level netlist is functionally identical to the original Golden RTL, ensuring no logic was broken during synthesis optimizations.

Tools

- Cadence Xcelium
- Cadence Genus
- Cadence Conformal

Instructions

The students are required to design and submit the following files for evaluation of this lab which is given below:

- The report should be in detail. Each step and all the commands should be explained properly. Also, the results and its effects should be explained.
- Your files should be submitted in the format, i.e.
 - Name – Module – Lab Number.zip [Includes all required sv, rpt, sdc, tcl files]
 - Name – Module – Lab Number.pdf [Your Report]



1. Introduction

Logic Synthesis is the automated process of translating abstract, human-readable hardware code (RTL, like Verilog or SystemVerilog) into a structural, gate-level netlist tied to a specific foundry manufacturing technology (like TSMC 65nm).

It is the definitive bridge between software behavior and physical silicon. The synthesis engine (like Cadence Genus) executes this in three primary phases:

- **Translation:** The tool reads the RTL and converts the if/else statements and always blocks into generic, unoptimized Boolean equations (AND, OR, NOT).
- **Optimization:** The mathematical equations are minimized to remove redundant logic and reduce the total gate count.
- **Technology Mapping:** The generic math is replaced with actual physical standard cells from the foundry's .lib files.

Throughout this process, the synthesis engine constantly calculates tradeoffs to meet your design constraints, aiming for the perfect balance of **PPA**:

- **Performance** (Meeting the clock speed/timing constraints).
- **Power** (Minimizing leakage and dynamic power).
- **Area** (Using the smallest standard cells possible).

2. Learning Objectives

After completing this lab on Advanced ASIC Front-End Synthesis, you will be able to:

- **Translate** abstract RTL (Verilog) into a structural, gate-level netlist mapped to a specific foundry's technology library (e.g., TSMC 65nm).
- **Differentiate** between purely Logical Synthesis and Physical-Aware (iSpatial) Synthesis, explaining the impact of physical wire RC delays on timing closure.
- **Configure** a Multi-Mode Multi-Corner (MMMC) timing environment to simultaneously optimize constraints for both high-speed functional operation and slow-speed test modes.
- **Implement** Design-for-Test (DFT) logic by inserting muxed scan chains and evaluate the physical area and timing penalties introduced by testability.
- **Perform** post-synthesis dynamic simulation using Standard Delay Format (SDF) in Cadence Xcelium to verify the physical timing and functional correctness of the netlist.
- **Perform** Formal Verification (LEC) using Cadence Conformal to mathematically prove that the structural gate-level netlist is functionally identical to the original Golden RTL, ensuring no logic was broken during synthesis optimizations.
- **Analyze** synthesis reports (Timing, Area, Power, and QoR) to critically evaluate the physical and architectural tradeoffs made by the synthesis engine.

3. The Paradigm Shift: Logical vs. Physical Synthesis

To understand why modern flows, require advanced tools, it is crucial to recognize the differences between these two methodologies. As transistors have shrunk, the delay



caused by the physical copper wires has surpassed the delay of the logic gates themselves.

Here is how the synthesis engine evolves to handle this reality:

Feature	Logical Synthesis (Basic Flow)	Physical-Aware Synthesis (Advanced/iSpatial)
Primary Focus	Mathematical boolean optimization and simple gate sizing.	Silicon floorplanning, gate placement, and real-world wire physics.
Wire Delay Estimation	Uses Wire Load Models (WLM) , which are statistical, highly inaccurate guesses based purely on fan-out.	Uses Virtual Routing , measuring actual 2D/3D physical distances between placed cells on the die.
Required Libraries	Only logical timing and power files (.lib).	Timing (.lib), Physical Geometry (.lef), and RC Interconnect Physics (.captable or .tch).
Constraints Setup	Standard SDC constraints forced directly onto the logic paths.	MMMC Setup , requiring the tool to balance multiple extreme temperature and voltage realities simultaneously.
Tool Execution	Elaborates RTL -> Minimizes Boolean Logic -> Maps to tech gates.	Maps gates -> Generates virtual floorplan -> Places cells -> Physically moves gates to shorten long wires.
Timing Correlation	Poor for deep-submicron nodes (65nm and below). Timing often fails once actual wires are added to the layout.	Excellent. The timing reports in synthesis closely match the final post-layout timing in Cadence Innovus.
Tool Handoff (Export)	Passes a Gate-Level Netlist (.v) and an SDC file to the layout tool.	Passes the Netlist, SDC, and a Physical Placement Database (.db / .def) containing the exact X/Y coordinates of every gate.

Table 1: Comparison of Logical vs. Physical Aware Synthesis

4. The Anatomy of Synthesis: Understanding the Input Files

Before launching the synthesis engine, it is critical to understand the different languages and files required to translate abstract RTL into physical silicon. In an



industry environment, you will rarely use a graphical user interface (GUI) to click through these steps; instead, you will use a combination of the following files to define the rules of the physical world.

4.1. The Automation Engine: TCL Scripts (.tcl)

Tool Command Language (TCL) is the scripting language that powers almost all modern EDA tools, including Cadence Genus and Innovus. Instead of manually typing commands one by one, we use a "Master Script" to automate the entire flow. This script defines directory paths, loads the design files, applies constraints, executes the synthesis mapping, and exports the final reports. Using TCL ensures your synthesis runs are highly reproducible and professional. The details of tcl is given below:

Setup & Library Initialization	
Command	Description
read_mmmc	Reads the Multi-Mode Multi-Corner (.view) setup file. This is the cornerstone of physical synthesis, linking the SDC constraints to real-world RC parasitics and temperature extremes.
read_physical -lef	Loads the Library Exchange Format (.lef) files. It provides Genus with the physical 2D dimensions and pin locations of the standard cells so it can build a virtual floorplan.
set_db init_lib_search_path	Defines the directory path where Genus should look for timing libraries.
read_libs	Loads the logical timing and power mathematical models (.lib) for a purely logical synthesis flow.

Table 2: Setup & Library Initialization

RTL Processing & Constraining	
Command	Description
read_hdl	Reads the Verilog source code files into the tool's memory. The -v2001 flag ensures compatibility with IEEE 1364-2001 Verilog syntax.
elaborate	The first major engine step. Translates the behavioral Verilog (e.g., if/else, always blocks) into a generic, unoptimized database of boolean logic equations.
check_design	Scans the elaborated design for fundamental coding errors, such as



	unresolved module instantiations or undriven input pins.
init_design	A powerful physical flow command that initializes the elaborated design by merging it with the loaded MMMC views and physical LEF data.
read_sdc	Applies the Synopsys Design Constraints (the timing budget and clock speeds) to the design in a basic logical flow.
check_timing_intent	Analyzes the loaded constraints to ensure they are mathematically sound and flags any logic paths that are left unconstrained.

Table 3: RTL Processing & Constraining

DFT Automation	
Command	Description
set_db dft_scan_style	Instructs the tool on which test architecture to build. muxed_scan tells Genus to replace standard flip-flops with scan-flops containing built-in multiplexers.
define_shift_enable	Physically creates the Scan Enable (SE) port on the top module, which acts as the toggle switch between Functional Mode and Test Mode.
check_dft_rules	Analyzes the RTL to verify it is legally testable. It ensures the external tester machine has direct, unblocked control over all clock and reset pins.
define_scan_chain	Defines the physical architecture of the shift register, explicitly naming the chain and auto-creating the scan_in and scan_out top-level ports.
connect_scan_chains	Physically stitches the test structure together, routing the Q output of one scan-flop to the SDI (Scan Data In) pin of the next flop.

Table 4: DFT Automation

The Core Synthesis Pipeline	
Command	Description
syn_generic	Translates the elaborated design into a technology-independent database



	and performs mathematical boolean optimization to minimize the logic
syn_map	Physically maps the optimized, generic logic structures to the actual physical standard cells provided in the foundry's technology library (.lib)
syn_opt	Performs incremental optimization by dynamically resizing gates, restructuring logic paths, and inserting buffers to resolve any remaining timing, area, or Design Rule Violations (DRVs).
write_do_lec	Automatically extracts the current state of the netlist and generates a complete Cadence Conformal .do script. It is run twice: once after syn_map to verify the baseline RTL-to-Gate translation, and once after syn_opt to prove that aggressive timing optimizations did not alter the Boolean logic.

Table 5: The Core Synthesis Pipeline

Reporting & Handoff (Export)	
Command	Description
report_*	A family of commands (report_timing, report_area, report_power, report_qor, report_scan_chains) that generates human-readable text summaries of the chip's physical and timing status.
write_hdl -mapped	Exports the final, synthesized, gate-level Verilog netlist to be used in simulation and physical layout.
write_sdf	Exports the Standard Delay Format (.sdf) file, which contains the exact picosecond timing delays of every mapped gate and physical wire for dynamic verification in Xcelium.
write_sdc	Exports an updated constraints file containing any newly generated logic paths (like the scan chains) for the Back-End tool.
write_db	Saves the entire physical layout state (the virtual floorplan, cell placements, and routing estimates) for a seamless handoff to Cadence Innovus.



write_dft_atpg	Generates a complete directory containing setup files, pin maps, and test execution scripts designed specifically for the Cadence Modus ATPG tool.
write_scandef	Exports a ScanDEF file. This tells the physical router (Innovus) exactly how the scan chains are wired so the router doesn't accidentally disconnect them while trying to optimize regular functional paths.

Table 6: Reports & Handoff

4.2. The Design Rules: SDC Constraints (.sdc)

The Synopsys Design Constraints (SDC) file dictates the "timing budget" for the chip. Genus does not inherently know how fast you want your chip to run; you must explicitly tell it. The SDC file defines:

- The target clock frequency (e.g., 100 MHz).
- The expected delays of signals arriving from outside the chip (Input Delays).
- The time required for external devices to read your output signals (Output Delays).
- Physical margins of error (Clock Uncertainty/Jitter).

4.2.1. Clock Definition

The clock is the heartbeat of a digital circuit. These commands define its speed and prevent the synthesis engine from mishandling high-fanout nets before the Place & Route (Innovus) phase.

Command	Description
create_clock	Defines the target operating frequency (e.g., -period 10.0 for 100 MHz) and the duty cycle waveform. It physically attaches this mathematical clock model to your Verilog clk input port.
set_ideal_network	This command is crucial for Pre-Layout Synthesis. It instructs Genus to treat high-fanout nets (like clocks and resets) as "ideal" with zero delay. This prevents the engine from attempting to build a premature, unoptimized clock tree during the synthesis phase. Note that these ideal properties are replaced with real physical trees later during the implementation phase in Innovus.

Table 7: Clock Definition



4.2.2. Clock Uncertainty (Safety Margins)

In the real physical world, clock oscillators are not perfect. They suffer from physical imperfections. We use these commands to steal a fraction of time from the tool as a safety buffer.

Command	Description
set_clock_uncertainty -setup	Accounts for "Jitter" (clock period variation). If your period is 10ns and you set a 0.15ns uncertainty, Genus must now optimize the data to arrive in 9.85ns. It forces the tool to work harder to guarantee safety.
set_clock_uncertainty -hold	Accounts for "Skew" (clock arriving at different flip-flops at slightly different times). It forces the tool to guarantee data remains stable for a short period <i>after</i> the clock edge hits.

Table 8: Clock Uncertainty

4.2.3. Environment Modeling

Your synthesized chip will eventually be placed on a larger PCB or inside a larger SoC. Genus needs to know the physics of the environment *outside* your block so it can correctly size the logic gates *inside* your block.

Command	Description
set_driving_cell	Simulates the electrical strength of the gate driving your input pins from the outside world. If omitted, Genus assumes your inputs are pushed by an infinitely powerful signal, which is physically impossible.
set_load	Simulates the physical weight (capacitance in pF) that your output pins must push. If omitted, Genus assumes a 0 pF load and will use the weakest standard cell possible, which would fail to drive a real wire on the silicon.
set_input_transition	Simulates the slew rate (sharpness) of incoming signals. Real digital signals take time to ramp from 0V to 1V. A slow transition time forces Genus to build stronger receiver gates to prevent logic errors.



Table 9: Environment Modeling

4.2.4. I/O Timing Budgeting

Your chip does not get to use the entire 10ns clock period for its internal logic. Data takes time to travel to your chip, and external devices need time to process data coming from your chip.

Command	Description
set_input_delay -max	(Setup Timing) Subtracts the time the data spent traveling through external logic <i>before</i> reaching your chip. If the external delay is 1.0ns, your internal logic now only has 9.0ns left to safely capture it.
set_input_delay -min	(Hold Timing) Defines how quickly the external logic might change the signal after the clock edge. It guarantees the minimum amount of time the incoming data will stay stable.
set_output_delay -max	(Setup Timing) Subtracts the time the external receiving device needs to process your output. Your block must push the signal out early enough to give the external device time to catch it.
set_output_delay -min	(Hold Timing) Dictates how long your chip must hold the output signal stable after a clock edge so the external device doesn't capture corrupted data.

Table 10: I/O Timing

4.2.5. DRV Constraints

While timing constraints define the "when" of a signal, **Design Rule Violations (DRVs)** define the "how" of a signal electrical health. Even if a path meets its timing budget, it must follow these physical rules to ensure the chip is reliable, low-power, and immune to electronic noise. Adding these to the SDC forces Genus to resize gates or insert buffers to keep the signals "sharp" and the power grid stable.

Command	Description
set_max_transition	Limits the maximum time a signal takes to rise from 0 to 1 (or fall from 1 to 0). By keeping transitions sharp (e.g., < 0.5ns), we prevent "short-circuit power" where both transistors in a CMOS gate are partially ON, which wastes energy and generates excessive heat.



set_max_capacitance	Restricts the total physical "weight" (capacitance) that any single gate is allowed to drive. If a wire is too long or a gate is driving too many pins, the capacitance becomes too high, causing the signal to become "lazy" and vulnerable to noise.
set_max_fanout	Limits the total number of input pins that a single output pin can drive. In high-speed designs, driving too many gates (e.g., > 20) degrades the electrical integrity of the signal. Genus fixes this by automatically "cloning" gates to share the load.

Table 11: DRV Constraint

Note: You may add or remove parameters as needed. Nearly all parameters are shown in the figure below:

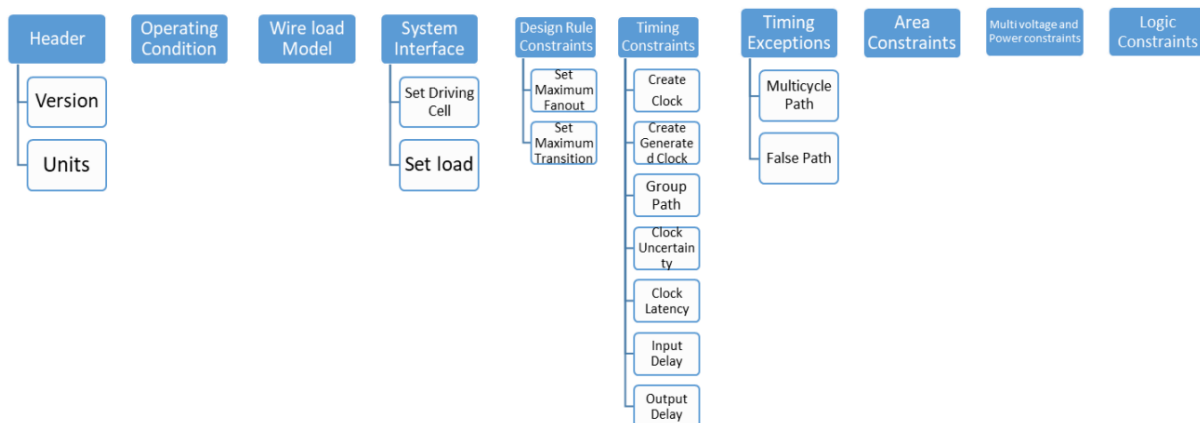


Figure 1: SDC Constraints

4.3. The Physical Reality: MMMC Setup (.view)

The Multi-Mode Multi-Corner (MMMC) setup file makes Physical-Aware Synthesis possible. It instructs the tool to optimize the chip across parallel realities. MMMC combines your design **Modes** (the SDC files) with physical **Corners** (extreme environmental realities). For example, silicon behaves very differently depending on the temperature. The MMMC file forces Genus to guarantee that your chip will successfully meet Setup Timing in a Worst-Case environment (e.g., 125°C, where transistors are slow) while simultaneously ensuring no Hold Timing violations occur in a Best-Case environment (e.g., -40°C, where data might shoot through the logic too quickly).

Command	Description
---------	-------------



create_library_set	Groups the mathematical timing and power models (.lib) for a specific foundry corner (Typical, Worst, or Best).
create_timing_condition	Wraps the library set into a formal "timing condition" object that the MMMC engine can utilize later in the flow.
create_rc_corner	Defines the physical wire physics of the environment. It specifies the operating temperature and loads the capacitance tables (.captable) so the tool knows exactly how much resistance the copper wires will have at that temperature.
create_constraint_mode	Defines a specific operational state by loading an SDC file. For example, func_mode loads the 100 MHz functional constraints, while test_mode loads the 20 MHz scan-shift constraints.
create_delay_corner	The first major merge. It combines the logical cell delays (timing_condition) with the physical wire delays (rc_corner) to create a complete physical reality (e.g., 125°C cells + 125°C wires = dc_worst).
create_analysis_view	The final structural merge. It combines <i>what</i> the chip is trying to do (constraint_mode) with the <i>physics</i> of where it is doing it (delay_corner). This creates a specific "universe" for the tool to analyze (e.g., view_func_setup).
set_analysis_view	The activation command. It tells the synthesis engine which views to actively use during optimization. Setup requires the slowest views (to ensure signals arrive in time), Hold requires the fastest views (to ensure signals don't disappear too quickly), and Power typically uses the baseline views.

Table 12: MMMC Commands



4.4. The Building Blocks: Technology Libraries (.lib, .lef, .captable)

When Genus translates your Verilog code, it must map your logic to actual physical "Lego bricks" manufactured by a specific foundry (e.g., TSMC). We provide three types of files to model these bricks:

- **.lib (Logical Timing & Power):** Defines the mathematical delay and power consumption of every standard cell (AND gates, Flip-Flops, etc.).
- **.lef (Library Exchange Format):** Defines the physical 2D footprint, pin locations, and metal layer rules for those cells. Without this, Genus cannot place the cells on a virtual floorplan.
- **.captable (Capacitance Table):** Defines the physical resistance and capacitance (RC parasitics) of the microscopic copper wires connecting the cells. This allows Genus to accurately calculate wire delays.

5. The Outputs of Synthesis

When Cadence Genus finishes running your Master Script, it does not just spit out a single file. It generates a complete package of hardware maps, physical databases, and testing instructions. These outputs are the critical handoff files required for the Back-End engineers (Place & Route).

Here is the breakdown of the files generated in your ./outputs, ./netlists, and ./reports directories:

5.1. The Gate-Level Netlist (.v)

This is the ultimate product of logic synthesis. It is a Verilog file, but it looks completely different from the RTL you wrote. The tool has stripped away every if/else statement and always block, replacing them entirely with a massive list of physical standard cell instantiations (e.g., XOR2D1, DFCNQD1) and the wires connecting them. The example of netlist file is given below:



```
// Generated by Cadence Genus(TM) Synthesis Solution 21.18-s082_1
// Generated on: Apr 29 2026 14:52:57 PKT (Apr 29 2026 09:52:57 UTC)

// Verification Directory fv/counter

module counter(clk, reset, count);
  input clk, reset;
  output [3:0] count;
  wire clk, reset;
  wire [3:0] count;
  wire n_0, n_1, n_2, n_3, n_4, n_5, n_6;
  DFCNQD1 \count_reg[3] (.CDN (n_6), .CP (clk), .D (n_5), .Q
    | (count[3]));
  DFCNQD1 \count_reg[2] (.CDN (n_6), .CP (clk), .D (n_3), .Q
    | (count[2]));
  MOAI22D1 g81__2398(.A1 (n_4), .A2 (count[3]), .B1 (n_4), .B2
    | (count[3]), .ZN (n_5));
  DFCNQD1 \count_reg[1] (.CDN (n_6), .CP (clk), .D (n_1), .Q
    | (count[1]));
  OA21D1 g83__5107(.A1 (n_2), .A2 (count[2]), .B (n_4), .Z (n_3));
  ND2D1 g85__6260(.A1 (n_2), .A2 (count[2]), .ZN (n_4));
  IA021D1 g86__4319(.A1 (count[0]), .A2 (count[1]), .B (n_2), .ZN
    | (n_1));
  AN2XD1 g88__8428(.A1 (count[0]), .A2 (count[1]), .Z (n_2));
  CKND0 g89(.I (reset), .ZN (n_6));
  DFCND1 \count_reg[0] (.CDN (n_6), .CP (clk), .D (n_0), .Q (count[0]),
    | .QN (n_0));
endmodule
```

Figure 2: Gate-Level Netlist

5.2. The Standard Delay Format (.sdf)

While the netlist tells the simulator *what* gates are connected, the SDF file tells the simulator *exactly how fast* those gates and wires are. It contains a massive list of picosecond-accurate real-world delay values calculated by the MMMC engine. The example of an SDC file is given below:



```

Open  counter_map.sdf
~\HIRA_PD_cadence\COHORT3\Frontend\out

(COND A1==1'b1&&A2==1'b1&&B2==1'b1 (IOPATH B1 ZN (::36) (::33)))
)
)
)
(CELL
(CELLTYP "DFCNQD1")
(INSTANCE count_reg\[1\])
(Delay
(ABSOLUTE
(PORT CDN (::0.0))
(PORT CP (::0.0))
(PORT D (::0.0))
(COND CP==1'b0&&D==1'b0 (IOPATH CDN Q () (::29)))
(COND CP==1'b0&&D==1'b1 (IOPATH CDN Q () (::29)))
(COND CP==1'b1&&D==1'b0 (IOPATH CDN Q () (::29)))
(IOPATH CDN Q () (::29))
(COND CP==1'b1&&D==1'b1 (IOPATH CDN Q () (::28)))
(IOPATH CP Q (::102) (::112))
)
)
(TIMINGCHECK
(RECREM (posedge CDN) (COND D_SDFCHK (posedge CP)) (::65) (::82))
(RECREM (posedge CDN) (posedge CP) (::65) (::82))
(SETUPHOLD (negedge D) (posedge CP) (::15) (::34))
(SETUPHOLD (posedge D) (posedge CP) (::6) (::10))
(SETUPHOLD (negedge D) (COND CDN_SDFCHK (posedge CP)) (::15) (::34))
(SETUPHOLD (posedge D) (COND CDN_SDFCHK (posedge CP)) (::6) (::10))
)
)
)

```

Figure 3: SDF File

- **CELLTYPE "DFCNQD1"**: This is a **D-type Flip-Flop** with:
 - D = Data Input
 - CP = Clock (positive edge)
 - CDN = Asynchronous Clear (Active Low)
 - Q = Output
- **INSTANCE count_reg[1]**: This refers to the 1st bit flip-flop in a register called count_reg.
IOPATH CP Q (::102) (::112): This defines the **Clock-to-Q** delay. It tells the simulator that after a clock edge hits the CP pin, the output Q will transition after approximately **102 to 112 ps**.
- **COND ... (IOPATH CDN Q)**: These lines define **Recovery/Removal** or **Clear-to-Q** delays. For example, if the asynchronous clear (CDN) is asserted, the output Q will fall in roughly **28–29 ps**.
(COND CP==1'b0&&D==1'b0 (IOPATH CDN Q () (::29)))
...
(COND CP==1'b1&&D==1'b1 (IOPATH CDN Q () (::28)))
(IOPATH CP Q (::102) (::112))
- These are the most critical values for timing verification. They define the "sampling window" around the clock edge.
(SETUPHOLD (negedge D) (posedge CP) (::15) (::34))
(SETUPHOLD (posedge D) (posedge CP) (::6) (::10))

D Transition	Setup Time (ps)	Hold Time (ps)
Negedge (1→0)	15 ps before clk	34 ps after clk
Posedge (0→1)	6 ps before clk	10 ps after clk

Figure 4: Setup/Hold Time



5.3. The Layout Databases (.db)

When running Physical-Aware Synthesis, Genus performs a "rough draft" of the chip's physical layout (iSpatial). It exports these blueprints, so the Back-End tool (Cadence Innovus) doesn't have to start from scratch. You will observe these on your own.

5.4. The Synthesis Reports (.rpt)

These human-readable text files (i.e. area, timing, power, gates, QoR & scan_chains reports) are how the design engineer grades the synthesis engine's performance. They contain the final "PPA" (Power, Performance, Area) metrics. You will observe these on your own.

5.5. Formal Verification Scripts and Logs (.do / .log)

When write_do_lec is executed, Genus exports specific files to automate the Cadence Conformal tool:

- **Dofiles (.do):** Found in the outputs/ directory. These are Tcl scripts pre-configured with the exact library paths, datapath solvers, and mapping instructions required to compare two designs.
- **Verification Data (.fv.json):** A "cheat sheet" generated by Genus that tells Conformal exactly which flip-flops were renamed, cloned, or merged during optimization to prevent false equivalence errors.

6. The Engine in Action: Executing Synthesis with Cadence Genus

Now that the directory structure is organized, and the input files (TCL, SDC, MMMC, and Libraries) are staged, it is time to launch the synthesis engine. In this module, we will execute the Master TCL script to drive Cadence Genus through the three major phases of synthesis: Generic compilation, Technology Mapping, and Incremental Optimization. For this demonstration, we will run logical synthesis in which Genus will ignore wire resistance and physical floorplanning, giving us a "perfect world" baseline of how fast and small our logic can mathematically be. The steps are given below:

6.1. Invoking the Cadence Environment

For this purpose, we run the terminal and change the directory into a specific directory where csh file is present. The illustration is given below:

```
[cc@ncdc-0081 ~]$ cd ./Fawad_PD/example/workspace
[cc@ncdc-0081 workspace]$ csh
[cc@ncdc-0081 workspace]$ source cshrc
*****
*****      Welcome to Cadence Tools      *****
*****
[cc@ncdc-0081 workspace]$ █
```

Figure 5: Invoking Cadence Environment

6.2. Invoking the Cadence Genus

There are two distinct methods for invoking the Cadence Genus synthesis engine, depending on your workflow needs. By simply typing genus into the Linux terminal, you



enter **Interactive Mode**, which allows you to execute TCL commands one by one. This approach is highly recommended for beginners learning the flow or engineers actively debugging, as it lets you instantly observe the engine's behavior step-by-step. Alternatively, for automation and production, you can use **Batch Mode** by passing your entire TCL script to the engine at launch (e.g., `genus -f genus_master_script.tcl`). This instructs Genus to read and execute the complete script from top to bottom without human intervention, demonstrating the true power of EDA scripting by ensuring your synthesis runs are fast, efficient, and perfectly repeatable. The illustration on terminal is shown below:

```
[cc@ncdc-0081 ~]$ cd ./Fawad_PD/example/workspace
[cc@ncdc-0081 workspace]$ csh
[cc@ncdc-0081 workspace]$ source cshrc
*****
*****      Welcome to Cadence Tools      *****
*****
[cc@ncdc-0081 workspace]$ genus -f /home/cc/Fawad_PD/example/scripts/genus_master_script.tcl
```

Figure 6: Invoking Genus

After running this command, you will observe the important outputs.

6.3. Genus GUI

After running this command, the genus gui is opened which is given below:

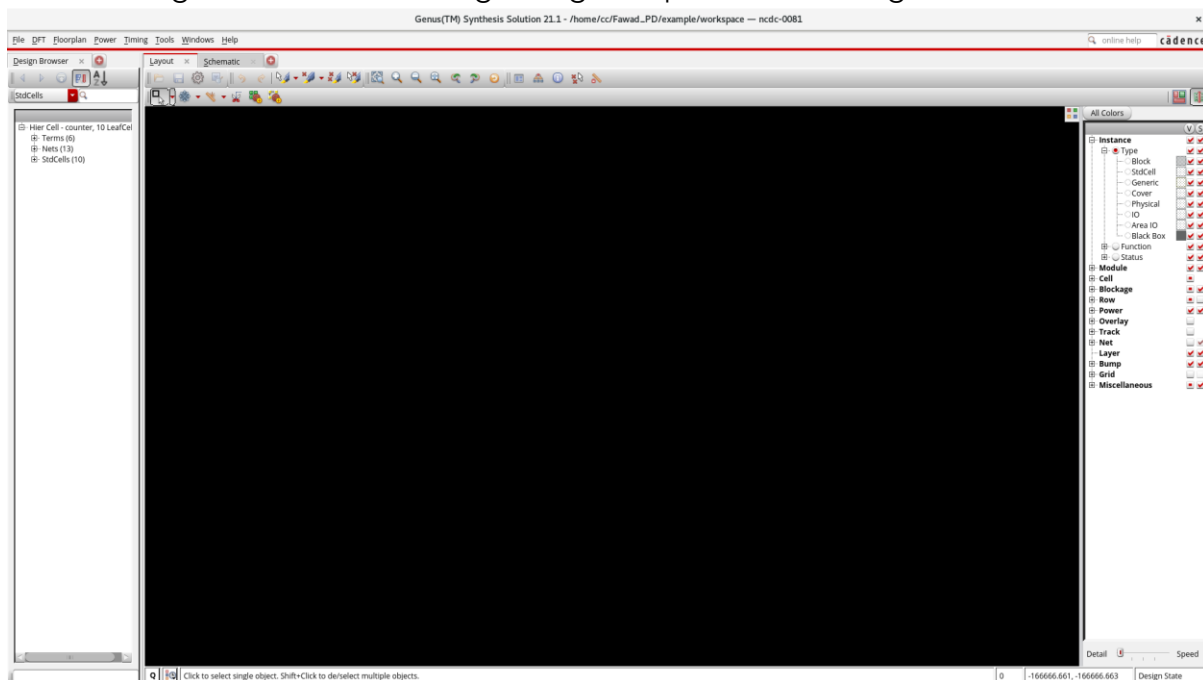


Figure 7: Genus GUI

By selecting the (+) button on the right side of layout tab, you will observe many options. Select the schematic option to open the schematic.

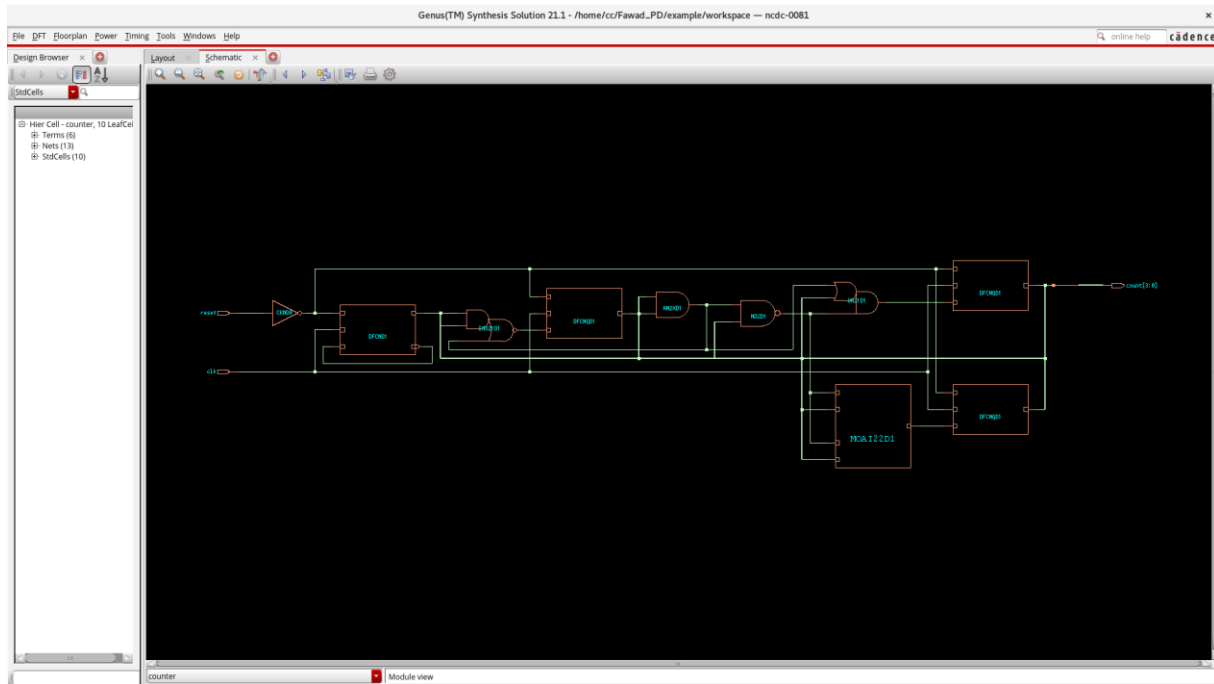


Figure 8: Schematic

As you observed, the schematic shows the circuit mapped onto technology-based standard cells.

6.4. Output Files

The required output files (i.e., netlist, .sdc, .sdf, and .rpt files) are automatically saved to a specific directory defined within the master script. The illustration is given below:

```
✓ db
  ≡ counter_synth.db
✓ netlists
  ≡ counter_physical_synth.v
✓ outputs
  ≡ counter_physical_synth.sdf
  ≡ mapped_to_opt.do
  ≡ rtl_to_mapped.do
✓ reports
  ≡ check_timing_intent.rpt
  ≡ counter_physical_area.rpt
  ≡ counter_physical_gates.rpt
  ≡ counter_physical_power.rpt
  ≡ counter_physical_qor.rpt
  ≡ counter_physical_timing.rpt
  ≡ counter_unconstrained.rpt
  ≡ lec_mapped_to_opt.log
  ≡ lec_rtl_to_mapped.log
```

Figure 9: Output Files



Observe each file & write in your lab report what the file tells us.

7. Gate-Level (Post Synthesis) Simulation

GLS is a step in the design flow to ensure that the design meets the functionality after synthesis or after placement and routing activities. Files required for post synthesis:

- Netlist File (.v)
- Testbench (.v)
- Sdf file (.sdf)
- Technology file (tcbn65lp.v)

Use the same test bench just add sdf annotation in it:

```
`timescale 1ns/1ps

module counter_tb;
reg clk;
reg reset;
wire [3:0] count;

    counter uut (
        .clk(clk),
        .reset(reset),
        .count(count)
    );

`ifdef SDF_TEST
initial
begin
    $sdf_annotate("/home/cc/Fawad_PD/example/workspace/outputs/counter_physical_synth.sdf", counter_tb.uut, "sdf.log", "MAXIMUM");
end
`endif
```

Figure 10: Testbench with SDF Annotation

In your terminal, run the script for post simulation in which flags are given below:

```
-timescale 1ns/1ps
/home/cc/Fawad_PD/example/workspace/netlists/counter_logical_synth.v
/home/cc/Fawad_PD/example/sourcecode/tb/counter_tb.v
-v
/home/cc/Fawad_PD/example/libraries/tcbn65lp.v
-access
+rwc
-define SDF_TEST
+sdf_verbose
-sdfstats /home/cc/Fawad_PD/example/workspace/reports/sdf_statistics.rpt
-maxdelay
-gui
```

Figure 11: Xcelium Required Commands for Post Simulation

After running this script, the waveform is given below:

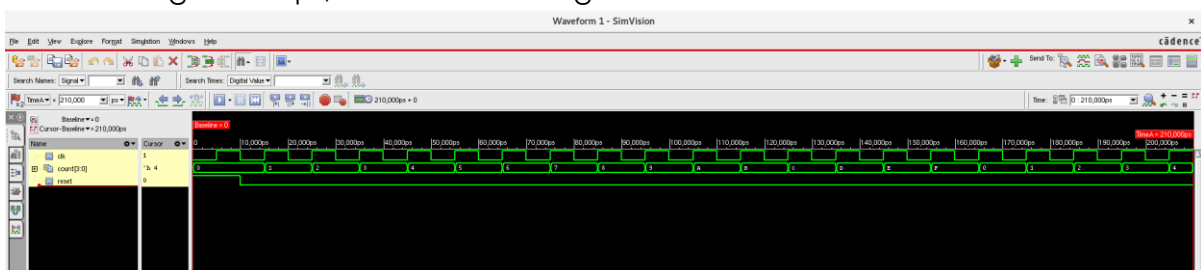


Figure 12: Post-Simulation Waveform



As you observed, there are no glitches or delays in the simulation as the synthesis is logical. Once physical synthesis is performed, you will observe the actual delays caused by RC & Standard Cells.

8. Formal Verification (Logic Equivalence Checking)

Simulation (GLS) checks if a design works by applying test vectors over time. Formal Verification (LEC) uses mathematical algorithms to prove that two versions of a design are 100% Boolean equivalent under *all* possible conditions, without needing a testbench.

In a professional Physical Design flow, we run LEC in a two-step defensive strategy:

- **RTL vs. Mapped (After syn_map):** Proves that the synthesis tool correctly translated the SystemVerilog RTL into generic physical standard cells without misunderstanding the code.
- **Mapped vs. Optimized (After syn_opt):** Proves that the aggressive timing and area optimizations (like deleting redundant gates or cloning high-fanout flops) did not break the fundamental logic.

8.1. Executing Cadence Conformal

Your master Genus script has automatically generated the two required .do files. To run the mathematical proofs, open your terminal and execute Conformal in Low-Power mode (-lp) using the following commands:

- **Run Check 1 (RTL vs. Mapped):**

```
lec -lp -dofile ./outputs/rtl_to_mapped.do -nogui
```

- **Run Check 2 (Mapped vs. Optimized):**

```
lec -lp -dofile ./outputs/mapped_to_opt.do -nogui
```

Now we dive into the 2nd Run Check which is Mapped vs. Optimized. You should perform 1st one also.

8.2. Deep Dive into Run Results: Check 2 (Mapped vs. Optimized)

Let's look at the exact terminal output generated by **Check 2** for your 4-bit counter design. This is the more fascinating of the two runs because it proves that the synthesis engine's aggressive optimizations worked flawlessly.

When you analyze an LEC log, you do not need to read every line. You are looking for these four specific milestones:

8.2.1. The Handshake (Verification Data)

```
Command: read_implementation_information fv/counter -golden fv_map -revised fv_opt  
Note: Implementation information 'fv_map.fv.json' will be used  
Note: Implementation information 'fv_opt.fv.json' will be used
```

What this means: Cadence Genus (the synthesis tool) and Cadence Conformal (the verification tool) are talking to each other. Genus left behind .json files that act as a "cheat sheet." If Genus renamed a flip-flop or merged two logic gates together to



save power, it wrote that action into the JSON file, so Conformal doesn't throw a false error when it sees the changed netlist.

8.2.2. The Proof of Optimization

Summary	Golden	Revised
Design-modules	1	1
Library-cells	9	8
Primitives	Golden	Revised
INPUT	2	2
OUTPUT	4	4
AND	2	2
DFF	4	4
INV *	6	5
XOR	3	3
Total	15	14

What this means: This is the physical footprint of your chip.

- **Golden (fv_map):** The baseline translation required 6 inverters (INV) and 15 total logic gates.
- **Revised (fv_opt):** The final optimized netlist requires only 5 inverters and 14 total logic gates. During the syn_opt step, Genus realized one of the inverters was logically redundant and deleted it to save physical silicon area and power.

8.2.3. The Mathematical Proof (The Compare Table)

Mapped points	PI	PO	DFF	Total
Golden	2	4	4	10
Revised	2	4	4	10
// ...				
Compared points	PO	DFF	Total	
Equivalent	4	4	8	

What this means: Even though Genus deleted an inverter and physically altered the architecture of your netlist, Conformal used mathematical proofs (Boolean logic) to map all 10 key endpoints together. When it ran the final compare command, it proved that the 4 Primary Outputs (PO) and the 4 Flip-Flops (DFF) behave the same way in both designs.



8.2.4. The Final Sign-Off (The Dashboard)

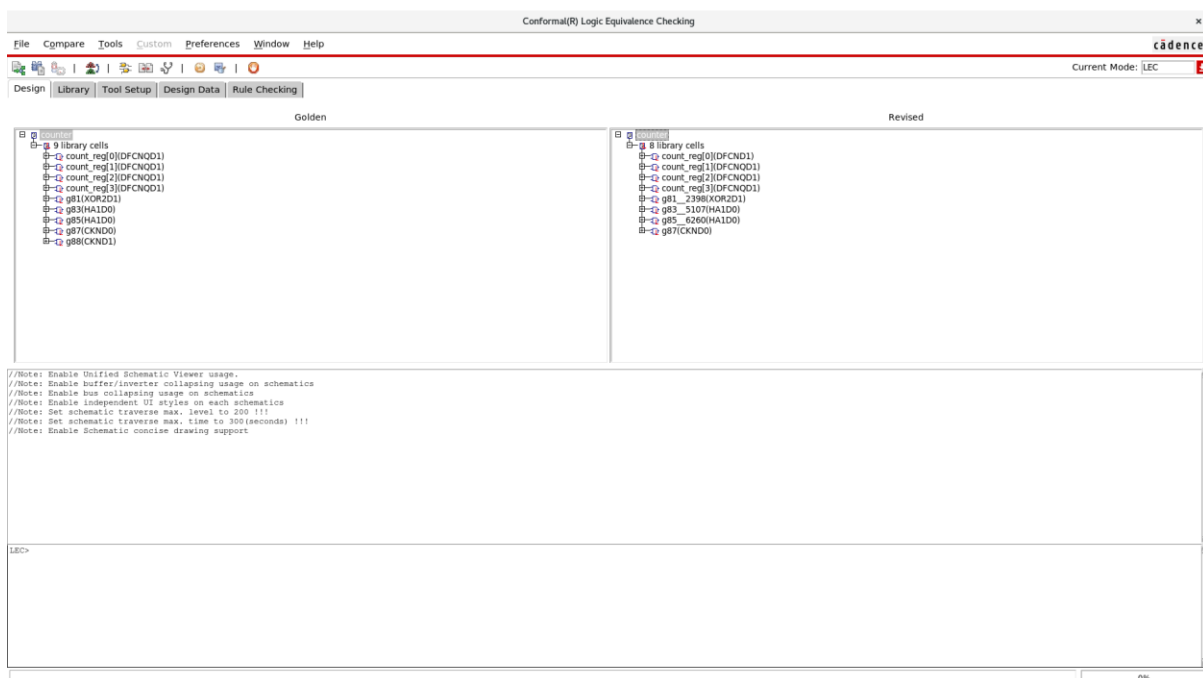
Conformal Version	23.10-s300
Root Module	counter
CPU Time	1 sec
Elapsed Time	1 sec
Memory	146.32 MB
LEC Comparison	PASS
Flatten Comparison	PASS

What this means: In the industry, this is the only table that matters. It is a clean dashboard confirming that the **LEC Comparison** is a definitive **PASS**. The gate-level netlist is officially verified.

8.2.5. Visual Debugging: Exploring the Conformal GUI

While running formal verification silently in the background (-nogui) is highly efficient for clean runs, debugging non-equivalent points or unmapped flip-flops requires a visual schematic. If your script fails and you need to physically trace a logic cone to see where Genus broke the math, you must transition into the Graphical User Interface (GUI).

To launch the graphical environment directly from an active Conformal terminal (LEC>), enter the command set gui on



Once the interface opens, you can utilize the **Mapping Manager** and the **Schematic Viewer** to visually explore the design, allowing you to trace the Golden RTL paths directly alongside the Revised Gate-Level Netlist. You can explore yourself.



9. Design for Test (DFT)

Design for Test (DFT) is a specialized engineering methodology that integrates auxiliary hardware circuitry into a chip's architecture during the synthesis phase to facilitate post-manufacturing structural verification. As modern integrated circuits scale to millions of transistors, physical probing of internal nodes becomes impossible; thus, DFT becomes a mechanical necessity to transform the design from a "black box" into a transparent, observable system. By replacing standard registers with **Scan Flip-Flops** and stitching them into serial **Scan Chains**, synthesis tools like Genus allow an Automatic Test Equipment (ATE) machine to shift in known data patterns and observe the results, effectively catching manufacturing defects like "stuck-at" faults or open circuits. Without the early integration of DFT in the synthesis pipeline, a design might be logically perfect yet physically un-verifiable, leading to massive financial losses if defective silicon cannot be identified and discarded before reaching the end of customer.

9.1. Flow of DFT

Here is the flow of DFT which is given below:

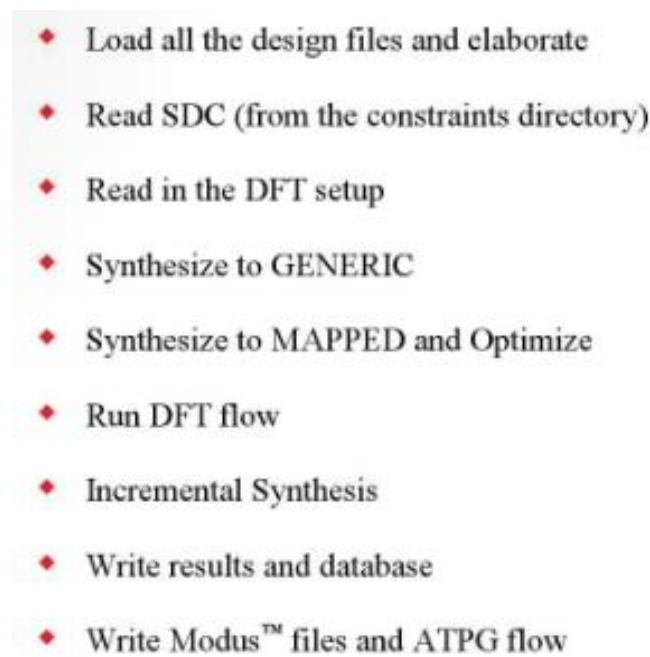


Figure 12: DFT Flow

9.2. The Anatomy of DFT: Building the "Safety Net" for Silicon

In this section of the lab manual, we explore how Genus automates the insertion of test structures.

9.2.1. DFT Automation & Scan Chain Insertion

In a professional DFT flow, we don't manually wire test logic. We use a set of high-level commands to instruct Genus to "stitch" the design together into a massive shift register known as a Scan Chain. These commands are covered in Table 4.



9.2.2. DFT Environment: The Test SDC (.sdc)

The **DFT Environment** is established by creating a specialized set of timing constraints, known as the **Test SDC**, which defines the operational rules for the chip while it is connected to industrial testing machinery. Unlike the functional environment that prioritizes high-speed performance, the test environment operates at a significantly slower clock frequency to ensure that data can be reliably shifted through the long, serial scan chains without corruption. This mode utilizes a "Case Analysis" technique to logically lock the chip into its test state, effectively bypassing the complex functional logic of the counter and focusing the synthesis engine's attention purely on the timing of the shift register paths. By isolating the test structures from the normal operational logic and defining specific timing budgets for the tester's interface, the synthesis tool can guarantee that the scan chains will function correctly on the manufacturing floor, even in the presence of physical parasitics and clock skew.

9.2.3. DFT Handoff: Reports & Exports

Once the scan chains are stitched, we must export specific "map files" so the next tools in the chain (Innovus for Layout and Modus for Pattern Generation) know what we built. These commands are covered in Table 6.

9.2.4. DFT in the Multi-Mode Multi-Corner (MMMC) Reality

In a DFT-enabled flow, the MMMC setup expands the tool's perspective by introducing **Test Analysis Views** alongside standard functional views. This multi-mode approach is essential because test hardware (the scan chains) often introduces a timing penalty on the functional paths. By defining a dedicated **Test Constraint Mode** that utilizes a much slower clock and a specific "Shift" configuration, the MMMC file allows the synthesis engine to optimize the serial scan paths independently. This ensures that the chip can transition seamlessly between its high-speed mission mode and its slow-speed diagnostic mode without either environment compromising the other's physical integrity across extreme temperature and voltage corners.

Command	Description
create_constraint_mode	This command defines a specific "goal" for the tool. While func_mode uses the standard SDC, the test_mode loads the specialized DFT SDC. This tells the tool: "Here are the rules for when we are testing."
create_analysis_view	This command merges a Mode (what the chip is doing) with a Corner (the physics of the environment). By creating view_test_setup, you are telling the tool to check if the scan chain works at 125°C.
set_analysis_view	This is the final master command. By including both view_func_setup and view_test_setup in the setup list, you



	force Genus to balance the requirements of both universes at once.
--	--

Table 15: MMMC for DFT

9.3. Executing Synthesis with DFT

We will drive Cadence Genus through the standard synthesis pipeline while simultaneously inserting and optimizing the scan chain hardware. Unlike a purely functional run, this flow requires a specialized "DFT Stitching" phase and an additional incremental optimization pass to ensure that the added test logic does not degrade the performance of the functional counter. By toggling between logical and physical modes, we can observe how the tool handles scan chain routing when aware of real-world wire lengths versus a generic mathematical model.

9.3.1. Invoking Cadence Genus

For this purpose, we will use the same genus master script and change the variable "INCLUDE_DFT" into "true". Run the commands one by one, observe the important outputs, and explain in your report. The output is given below:

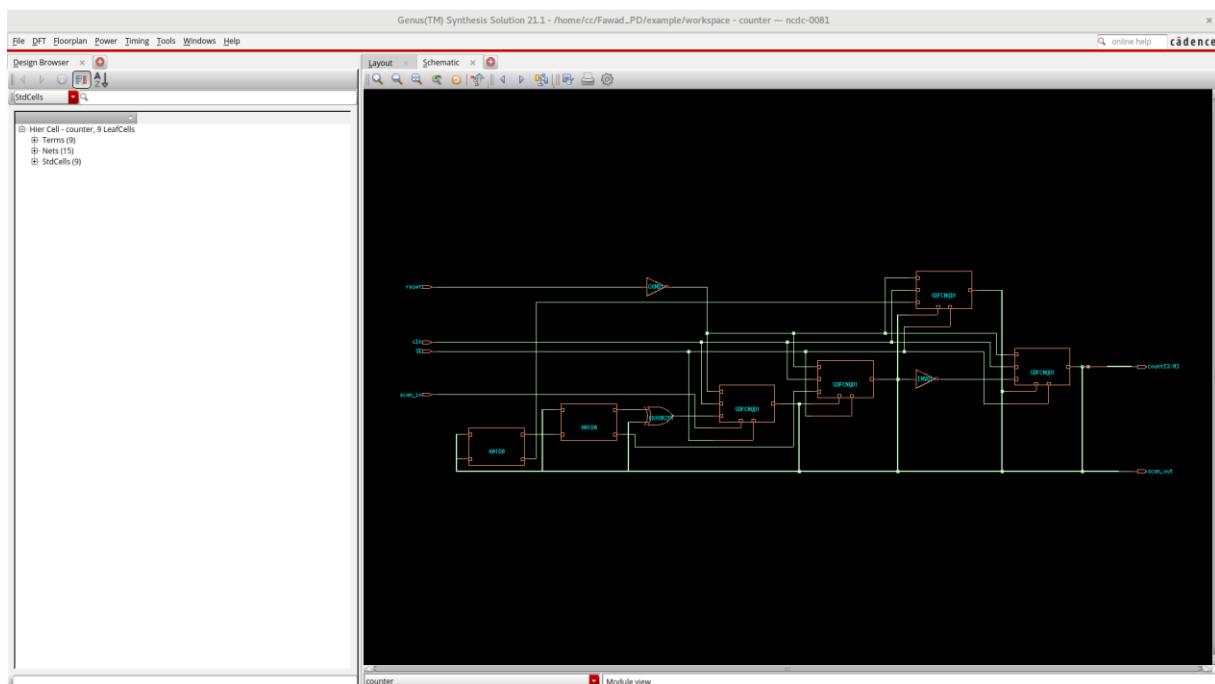


Figure 13: Schematic for DFT

As you observed that, the schematic is changed as some cells are changed i.e (simple FF changed into scanned FF).

9.3.2. Output Files

The required output files (i.e., netlist, .sdc, .sdf, and .rpt files) are automatically saved to a specific directory defined within the master script. The illustration is given below:

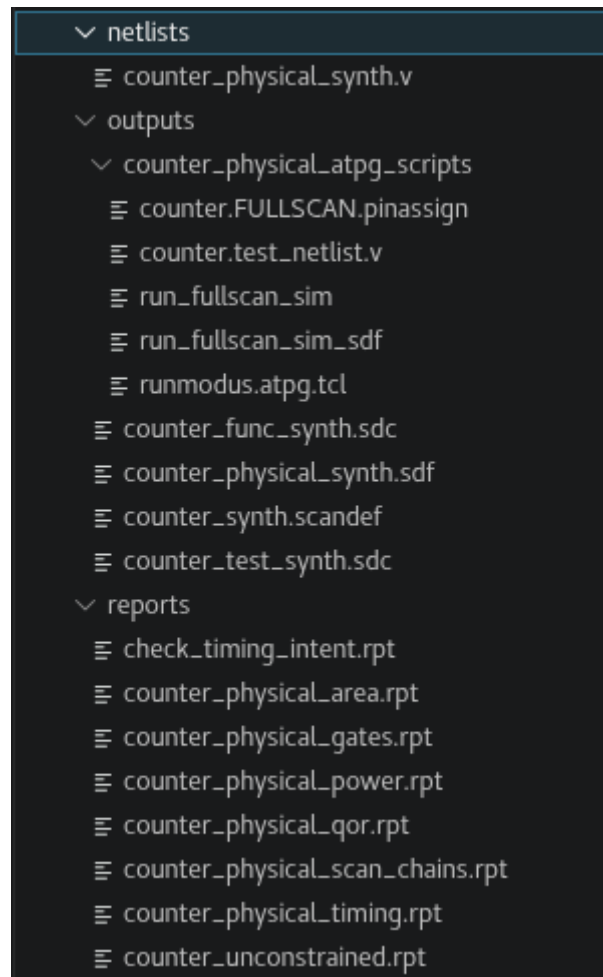


Figure 14: Output Files

9.4. Gate-Level (Post-Simulation) with DFT

To verify the design after scan insertion, the testbench must be updated to accommodate the newly created DFT ports: **scan_enable (SE)**, **scan_in (SI)**, and **scan_out (SO)**. The simulation is divided into two distinct operational phases. In **Functional Mode** (SE=0), the testbench verifies that the design still operates as a standard 4-bit counter without logic degradation. In **Shift Mode** (SE=1), the testbench provides a serial data pattern to the scan_in port and monitors the scan_out port to confirm that the internal flip-flops have been correctly stitched into a serial shift register. This verification ensures that the test hardware is physically functional and that the scan chains can successfully propagate data across the chip. The output is given below:

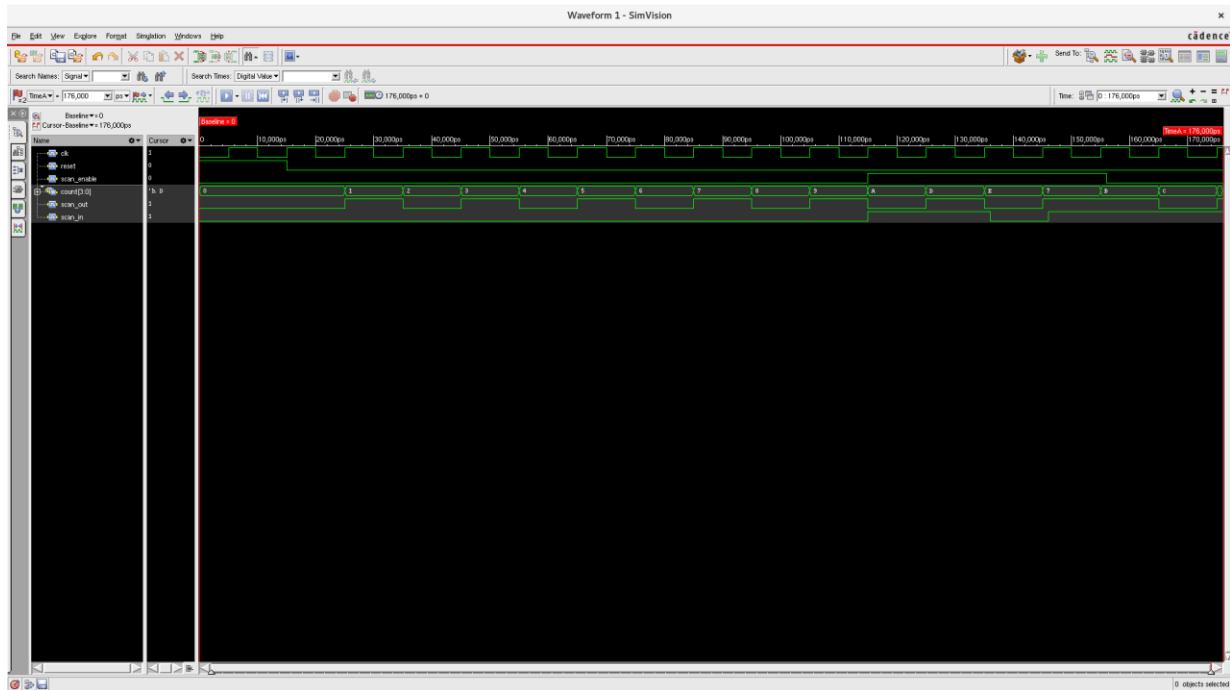


Figure 15: Post-Simulation with DFT

As you observed, the shift register works correctly, which means that the test hardware is physically functional.

10. Lab Task

Task 1: Perform both **logical synthesis** and **physical synthesis** of the 4-bit counter design using the provided scripts. Generate and analyze area, timing, and power reports for both flows.

- Summarize each output of each command.
- Document all generated files and explain their purpose.
- Write a brief analysis on how "Physical Awareness" (LEF and Capacitance Tables) affects the tool's ability to meet timing closure compared to the "Ideal" logical model.
- Run a **Post-Synthesis Gate-Level Simulation** (GLS) using the synthesized netlist and SDF to verify functionality.
- Provide screenshots of simulation and explain the results.

Task 2: Perform Formal Verification (LEC)

- Execute both the `rtl_to_mapped.do` and `mapped_to_opt.do` scripts using Cadence Conformal.
- Review the generated log files in the `reports/` directory.
- In your lab report, take a screenshot of the final "Mapped Points" and "Compared Points" tables from the terminal output.



- Explain why the number of physical instances (like Inverters or Flip-Flops) might differ between the fv_map design and the fv_opt design, even though Conformal proves they are mathematically equivalent.

Task 3: Perform **DFT-based Physical Synthesis** by enabling the **INCLUDE_DFT** toggle in your master script. Generate and analyze area, timing, and power reports.

- Summarize each output of each command.
- Document all generated files and explain their purpose.
- Run a **Post-Synthesis Gate-Level Simulation**. Use the updated testbench to verify Functional Operation & Shift Operation.
- Provide screenshots of simulation and explain the results.

11. Resources

- https://youtube.com/playlist?list=PLZU5hLL_713zf_i38C7uLu5pUz5wTjKul&si=_sdIpTE48TWc2yPC
(Watch 1st 6 Videos Regarding Synthesis)
- https://www.youtube.com/watch?v=DbzeOrYz_A8